

Jubilo Event Scaling Recommendation

Oct 3, 2026 · @Ismael Baez

Both near-term events — the district event (~15 people) and the youth event (200–300 people, two weeks later) — run on what is already live in production; nothing more needs to change before either. The real scaling work is for the ~35,000-person convention in July 2027, and it is deliberately deferred to spring 2027.

Verified baseline (all live in production)

Every service runs on an 8 vCPU / 8 GB Railway allocation at one replica, and every number below was confirmed on the running production containers and databases, not read from config.

Service	Gunicorn workers	Why that number	Postgres max_connections	In use
jubilo-auth	12	Below the (2 × cores) + 1 rule of 17: introspect, token and login have short CPU-bound password-hashing bursts	500	~9
jubilo-music	16	Near the rule of thumb: purely I/O-bound in-request; audio processing runs on the separate RQ worker	500	~9
jubilo-church	16	Same as music: picture processing runs on the separate RQ worker	500 (raised from 100 with ALTER SYSTEM , then a restart)	~9

Worst-case concurrent database connections per service at one replica is about 20–25 (one short-lived connection per in-flight request, `CONN_MAX_AGE=0`), against a ceiling of 500. jubilo-auth's `/introspect` throttle is 3000/min, and its redundant inner client-secret check is cached for 10 minutes.

District event (~15 people, imminent): do nothing more

Fifteen people cannot stress any of the baseline above, so no change is needed before this event. Treat it as a live rehearsal of the youth event instead.

The one thing worth watching is behavior, not capacity: that photo uploads reach the gallery, and that nothing sits in `pending` with `retry_count=0` for more than 5 minutes — the exact fingerprint the picture reconciler exists to catch. A photo that does get stuck is the reconciler's cue, not a capacity problem.

Youth event (200–300 people, two weeks later): do nothing more, but know the one lever

This load sits comfortably inside the single-replica setup, so nothing needs to change in advance. If photo uploads visibly back up during the event, the lever is the replica count on the RQ worker service in the Railway dashboard — not the Dockerfile. `rqworker` has no workers flag; it is one process per replica.

Worker service	Safe to add replicas?	Why	Where to change it
jubilo-church	Yes	RQ's dequeue is atomic (<code>LPOP</code> / <code>BLPOP</code> / <code>LMOVE</code> , confirmed in the installed source), and the one real duplicate-processing risk — the stuck-picture reconciler re-enqueueing a merely slow job — is fixed: each enqueue records its real job ID, and the reconciler skips any job still queued, started, deferred or scheduled	Railway dashboard: Service → Settings → Replicas (its start command lives there too; there is no <code>railway.worker.json</code>)
jubilo-music	Yes	Audited: no path can create a duplicate job, the upload endpoint blocks a second in-flight submission per hymn, and it now has its own stuck-submission reconciler	Railway dashboard: Replicas

Worker service	Safe to add replicas?	Why	Where to change it
jubilo-auth	Leave at one	Its jobs (token-revocation broadcasts, participant mirroring) are light and infrequent, and it was not audited to the same depth — do that audit before scaling it; it should not need it	—

Each added replica opens its own database connections, which is irrelevant at two or three replicas against a ceiling of 500.

Convention (~35,000 people, July 2027): the real work, deferred on purpose

The honest bottleneck is auth, not the queues. At roughly 38 introspection requests per second — 35,000 attendees on 15-minute access tokens — every request still re-verifies the calling service's client secret through django-oauth-toolkit's outer `authenticate_client()` layer, which runs Django's deliberately expensive PBKDF2 hasher and is uncached. Only the inner duplicate check in jubilo-auth's own `post()` is cached today.

1. Revisit in April-May 2027, not before. The two near-term events never approach this load, so nothing here is worth touching for them.
2. Check whether django-oauth-toolkit has shipped a release containing PR #1877 (merged to master on 2026-10-02; jubilo-auth is pinned at 3.3.0). It records the authenticated client on the request, so the whole redundant re-check — and the cache around it — can be deleted.
3. If it has, adopt it as a real migration, not a version bump: it also introduces an explicit `can_introspect` capability on `Application` and refuses public clients, so `/introspect` needs a rewrite and a data migration.
4. If it has not, extend the existing cache to the outer `authenticate_client()` layer instead — same keying by client ID plus a hash of the provided secret, same 10-minute TTL.

5. Plan horizontal replicas across all three web services for the event itself. Each replica adds up to 16 concurrent database connections (sync workers, one connection per in-flight request) plus its worker, against a ceiling of 500 per database — so even eight replicas per service stay well clear.

Operational notes

Two things sit outside capacity but matter for every event: the reconcilers that catch a silently lost upload, and one mobile fix shipping today.

Reconciler	Run once, on the web service, after deploy	Re-run only when
jubilo-church pictures (<code>reconcile_stuck_pictures</code>)	<code>python manage.py bootstrap_picture_reconciler</code>	Redis loses its data (<code>FLUSHALL</code> , or the Redis service is re-provisioned)
jubilo-music hymn audio (<code>reconcile_stuck_hymn_audio_submissions</code>)	<code>python manage.py bootstrap_hymn_audio_reconciler</code>	Same

Each bootstrap only pushes the first job; the sweep then runs on the worker every 5 minutes and reschedules itself, and the schedule lives in Redis, so a worker restart self-heals. Running a bootstrap again while the chain is alive starts a second, redundant chain — harmless, but pointless. Both workers already run with `--with-scheduler` , which the rescheduling needs.

On mobile, the brief flash of cards repositioning as a scroll settled on the Administrations list (seen on a BLU G45) is fixed in the builds queued today — Android versionCode 30, iOS buildNumber 40 — by making every card one 16:9 frame. The one remaining variable-height list is my-photo-submissions; if it shows the same flash on a budget Android, the same fix applies: one row height.